

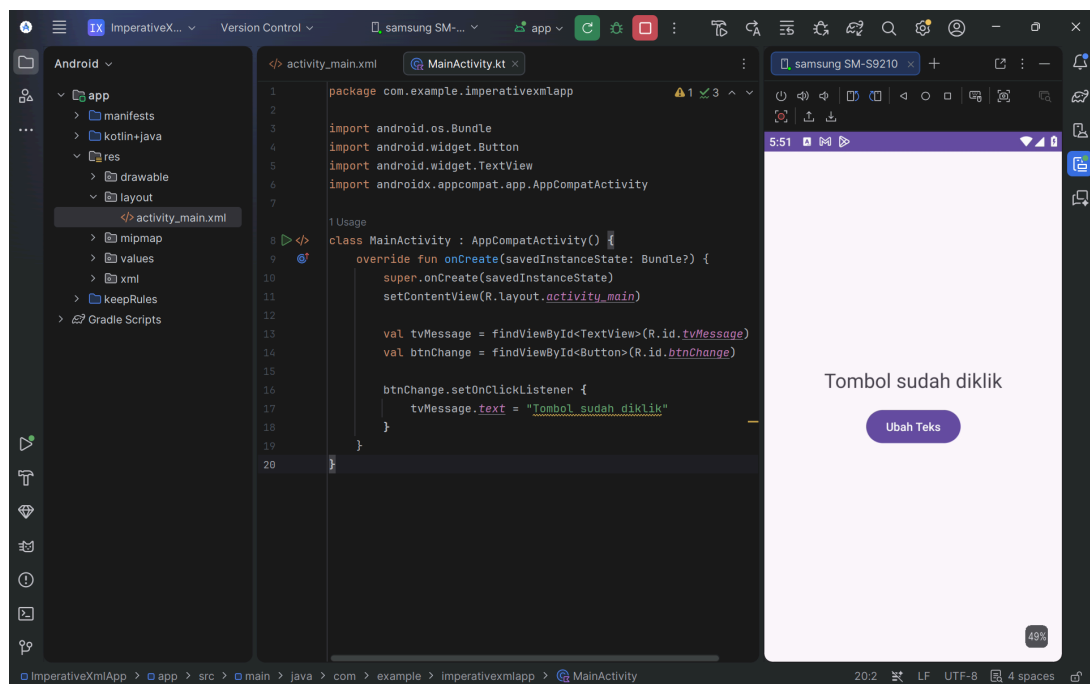
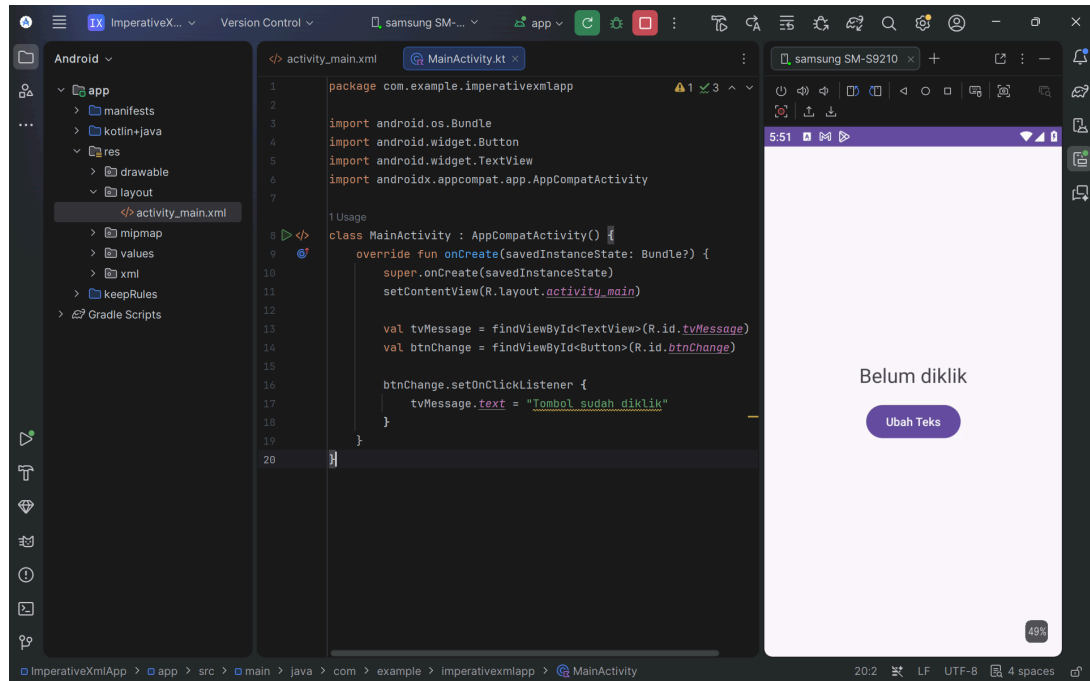
Nama : Ailsa Nasywa Hidayat
NIM : 245150407111052
Kelas : Pengembangan Aplikasi Mobile - D

MODUL 3

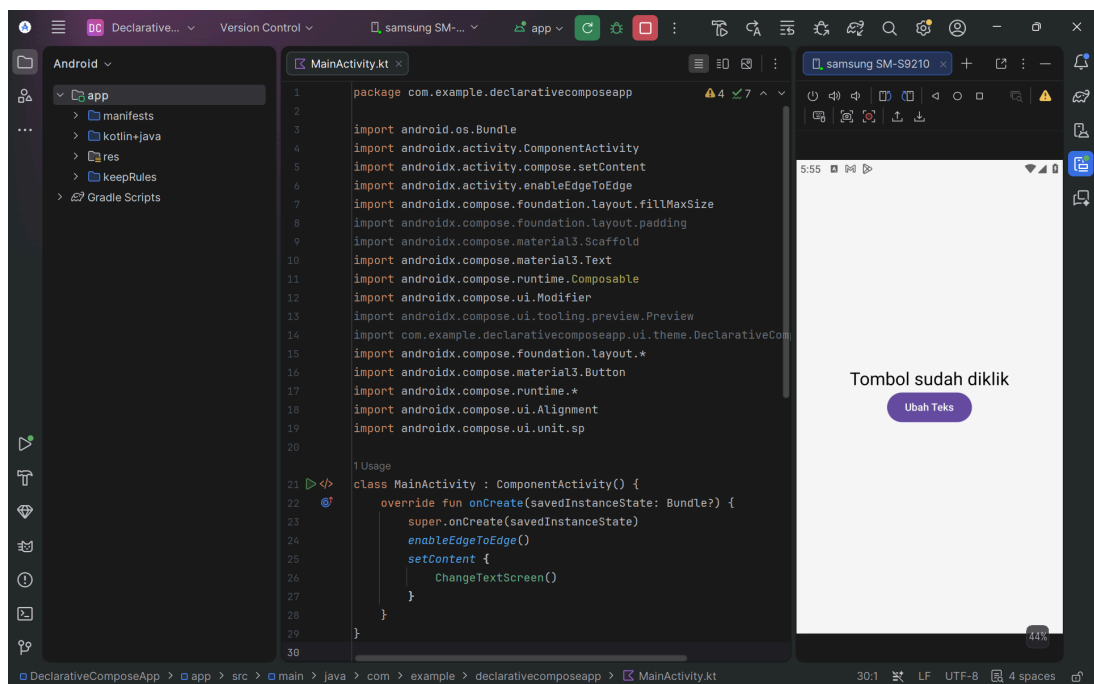
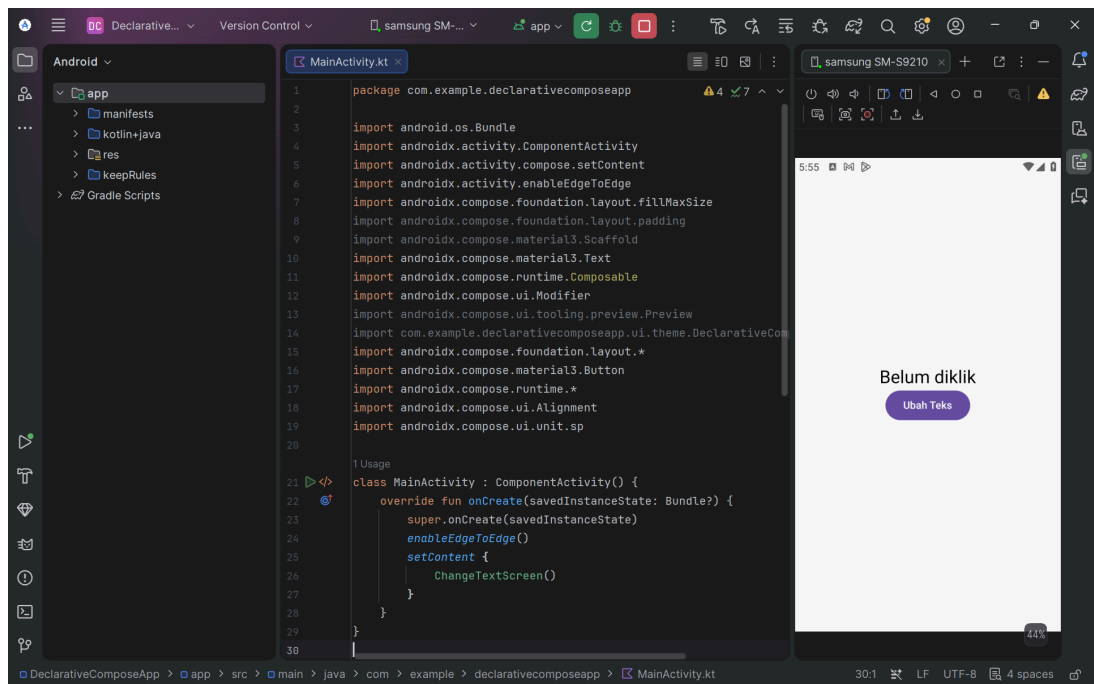
Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

1. Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik.

Project XML:



Project Compose:



2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.

Objek yang dimodifikasi langsung adalah TextView dengan variabel tvMessage. Ketika tombol ditekan, program mengubah isi teks pada komponen tersebut melalui:

```
btnChange.setOnClickListener {
    tvMessage.text = "Tombol sudah diklik"
}
```

Hal ini menunjukkan pendekatan imperatif karena program secara eksplisit menentukan komponen UI mana yang harus diubah dan perubahan apa yang harus

diberikan kepadanya. Kode XML/View pada modul menggunakan pola tersebut melalui findViewById() dan perubahan properti text.

3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.

Pada Compose, yang diperbarui bukan objek Text, tetapi state message:

```
Button(onClick = {  
    message = "Tombol sudah diklik"  
}) {  
    Text("Ubah Teks")  
}
```

State tersebut sebelumnya dibuat dengan:

```
var message by remember {  
    mutableStateOf("Belum diklik")  
}
```

Karena tampilan Text menggunakan nilai message, perubahan state akan menyebabkan isi yang ditampilkan ikut menyesuaikan.

4. Mengapa pada Compose tidak terdapat perintah seperti tvMessage.text atau setText()?

Jetpack Compose menerapkan pendekatan deklaratif, sehingga developer tidak perlu mengambil referensi komponen kemudian mengubah propertinya secara manual. Pada Compose, tampilan cukup didefinisikan berdasarkan state:

```
Text(text = message)
```

Ketika message berubah, Compose mengetahui bahwa bagian UI yang menggunakan nilai tersebut perlu diperbarui melalui recomposition. Karena itu tidak diperlukan perintah langsung seperti setText() atau tvMessage.text.

5. Jelaskan alur perubahan pada XML/View dengan pola: event -> View -> perubahan tampilan.

Alurnya dapat dijelaskan sebagai berikut:

Pengguna menekan tombol → event klik dijalankan → program mengakses TextView → properti teks diubah → isi yang terlihat pada layar berubah.

Pada pendekatan ini, perubahan UI dilakukan secara langsung terhadap objek tampilan.

6. Jelaskan alur perubahan pada Compose dengan pola: event -> state -> recomposition -> perubahan tampilan.

Alur pada Jetpack Compose adalah:

Pengguna menekan tombol → onClick berjalan → nilai state message diperbarui → Compose melakukan recomposition → Text membaca nilai state terbaru → tampilan berubah.

Jadi event pada Compose lebih berfungsi untuk mengubah data/state, sedangkan pembaruan UI kemudian ditangani oleh Compose.

7. **Apa yang terjadi jika message diganti menjadi variabel biasa: `var message = "Belum diklik"`? Jelaskan penyebabnya.**

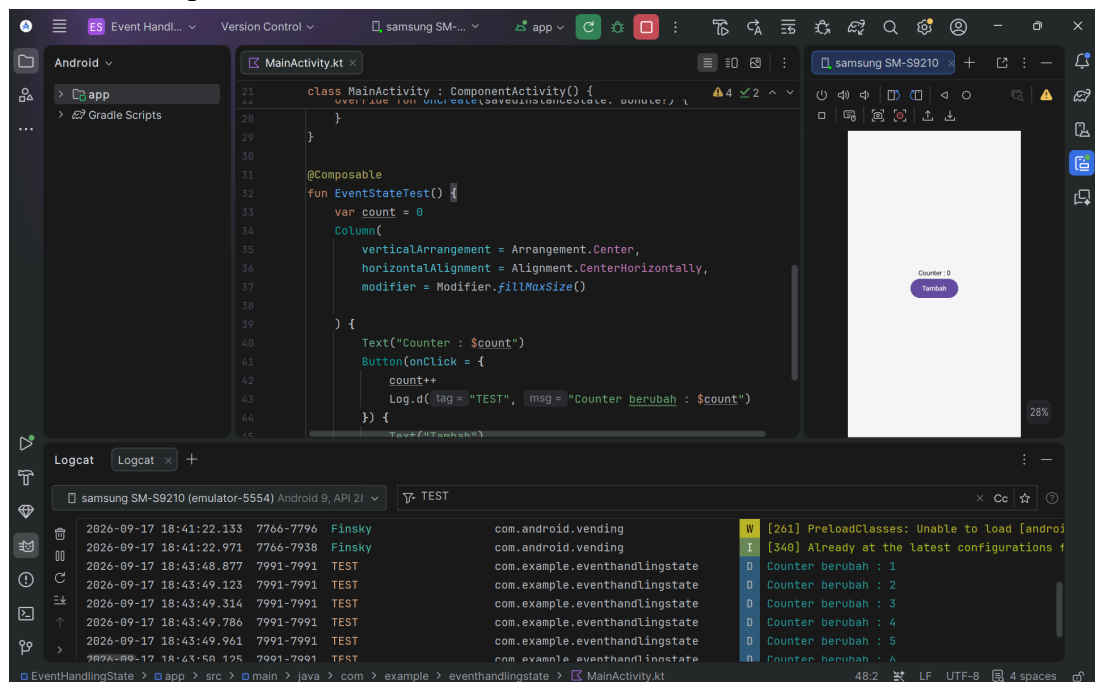
Jika ditulis seperti itu, nilai message memang dapat berubah ketika tombol ditekan, tetapi Compose tidak menganggapnya sebagai state yang perlu diamati. Akibatnya, perubahan tersebut tidak otomatis memicu recomposition, sehingga teks pada layar tidak dijamin ikut diperbarui. Agar perubahan data dapat diamati Compose, digunakan `mutableStateOf()` dan biasanya dipertahankan menggunakan `remember`.

8. **Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.**

Pada pendekatan imperatif, developer mengatur secara langsung langkah perubahan UI, misalnya mencari komponen tertentu lalu mengganti propertinya. Sementara itu, pada pendekatan deklaratif, developer menentukan bagaimana tampilan seharusnya terlihat berdasarkan state. Ketika state berubah, Compose menangani proses penyesuaian UI secara otomatis.

Event Handling State

1. **Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.**



2. **Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?**

Setiap klik menjalankan count++, LogCat mencatat perubahan nilai:

Counter berubah : 1

Counter berubah : 2

Counter berubah : 3

dst.

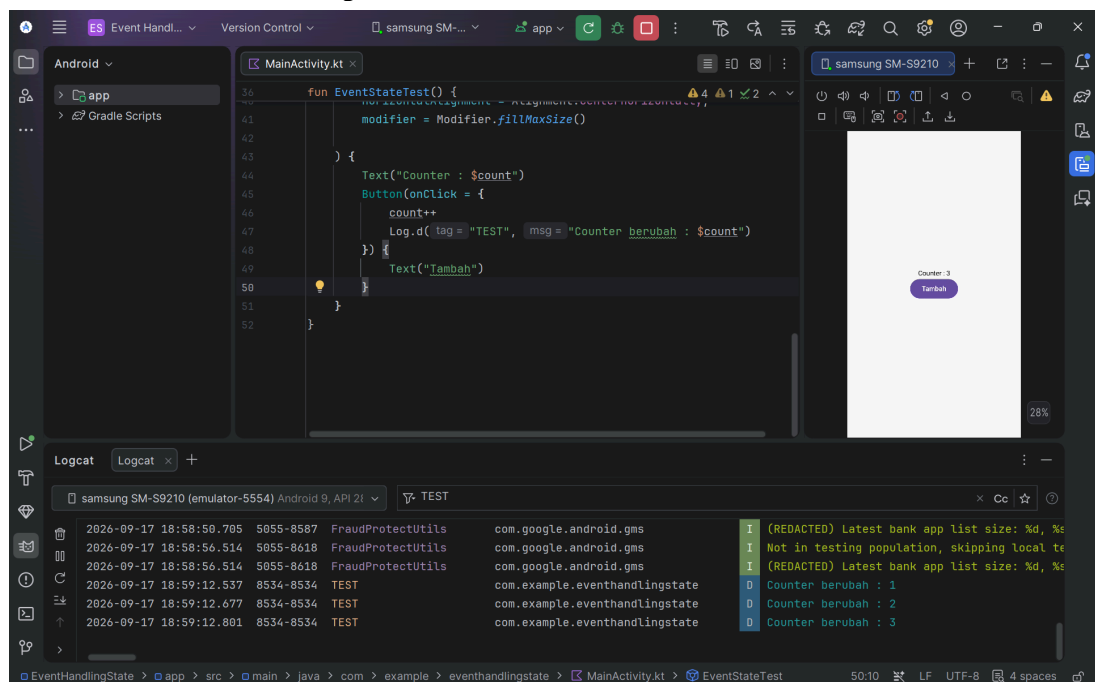
3. **Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?**

Walaupun nilai count berubah dan hasilnya tercatat di LogCat, teks counter pada layar tidak ikut memperbarui nilainya sebagaimana state Compose.

4. **Apa yang menyebabkan hal tersebut terjadi?**

Penyebabnya adalah count hanya merupakan variabel biasa. Perubahan variabel biasa tidak diamati oleh Compose. Karena tidak ada perubahan state yang dikenali Compose, proses recomposition untuk memperbarui teks counter tidak otomatis terjadi. Modul menjelaskan bahwa variabel lokal biasa dapat berubah, tetapi tidak otomatis meminta Compose menggambar ulang UI.

8. **Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.**



9. **Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?**

LogCat tetap menampilkan perubahan nilai counter setiap kali tombol diklik, seperti:

Counter berubah : 1

Counter berubah : 2

Counter berubah : 3

dst.

10. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Tampilan “Counter : 0” pada aplikasi ikut diperbarui berdasarkan perubahan count.

11. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

Pada percobaan kedua, count dibuat menggunakan:

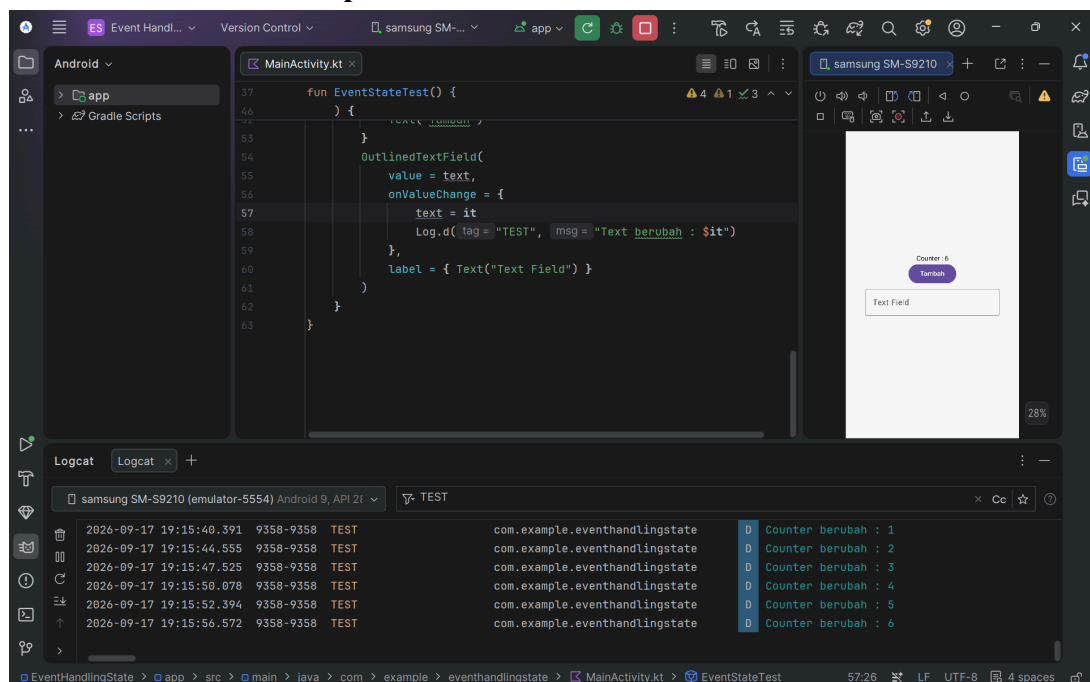
```
remember { mutableStateOf(0) }
```

sehingga count menjadi state yang dapat diamati Compose. Ketika “count++” mengubah nilai state, Compose mengetahui bahwa UI yang menggunakan count perlu diperbarui. Karena itu teks counter ikut berubah. Sebaliknya, pada percobaan pertama count hanyalah variabel biasa sehingga perubahannya tidak digunakan sebagai pemicu recomposition.

12. Apa yang dimaksud recomposition pada Compose?

Recomposition adalah proses Jetpack Compose memperbarui kembali bagian composable yang bergantung pada state yang mengalami perubahan. Pada percobaan ini, ketika count berubah, Compose mengevaluasi kembali bagian “Text("Counter : \$count)” sehingga nilai yang tampil selalu sesuai dengan nilai count terbaru.

14. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.

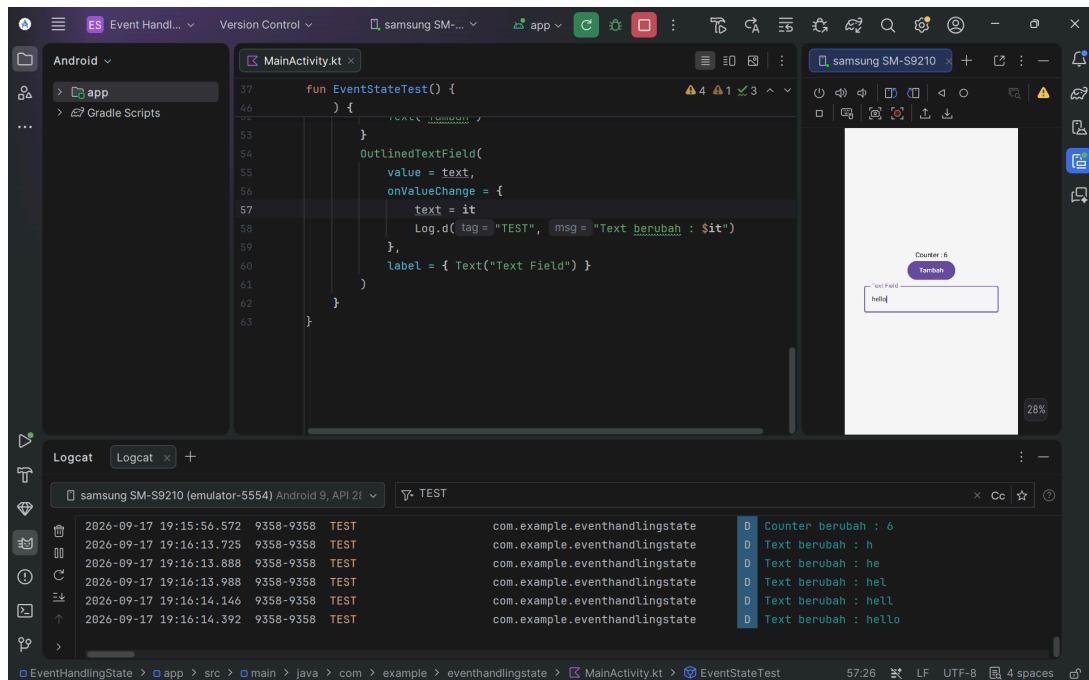


15. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?

Setiap perubahan isi input menjalankan:

```
Log.d("TEST", "Text berubah : $it")
```

Jika mengetik “hello”, LogCat akan memperlihatkan perubahan isi secara bertahap, misalnya:



16. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?

Isi `OutlinedTextField` berubah mengikuti karakter yang dimasukkan pengguna. Ketika pengguna menambahkan atau menghapus karakter, teks yang terlihat pada field langsung menyesuaikan dengan nilai terbaru dari state `text`.

17. Apa yang menyebabkan hal tersebut terjadi?

`OutlinedTextField` memiliki event:

`onChange`

Event tersebut dipanggil setiap kali isi field berubah. Nilai terbaru diberikan melalui `it` dan kemudian disimpan ke state:

```
text = it
```

Karena `value` pada `OutlinedTextField` berasal dari state `text`, perubahan state menyebabkan UI diperbarui sehingga input yang baru dimasukkan dapat terlihat.

18. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?

Terdapat dua state:

1) `count`

```
var count by remember { mutableStateOf(0) }
```

Digunakan untuk menyimpan nilai counter.

2) `text`

```
var text by remember { mutableStateOf("") }
```

Digunakan untuk menyimpan isi dari OutlinedTextField.

19. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?

Terdapat dua event utama:

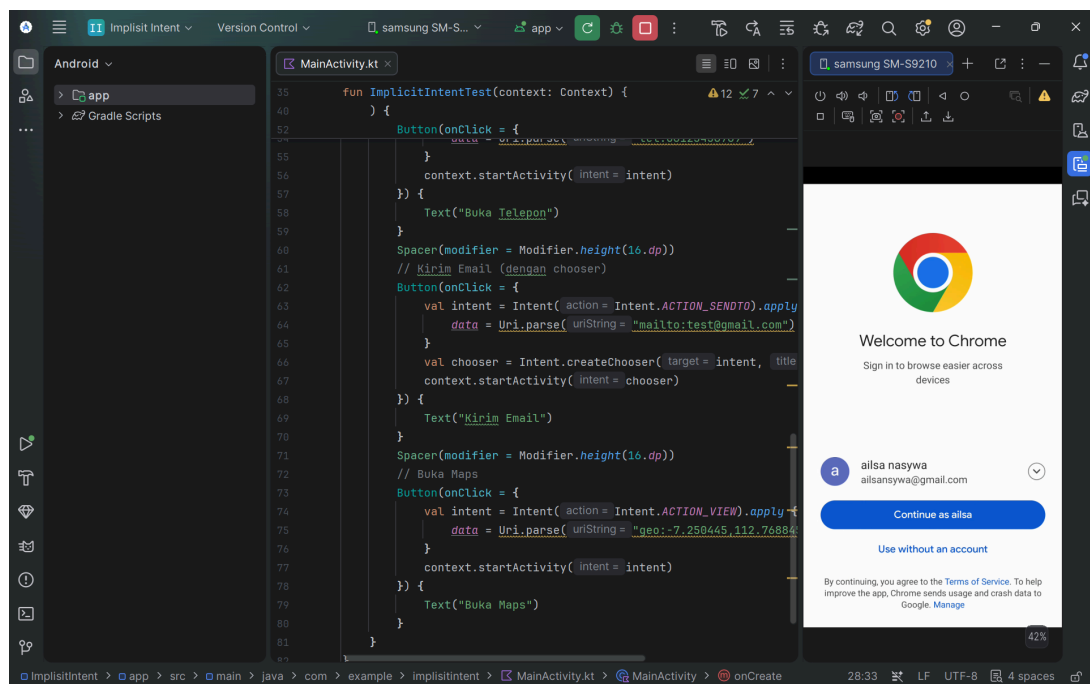
- 1) onClick, terdapat pada Button dan digunakan ketika pengguna menekan tombol Tambah.
- 2) onChange, terdapat pada OutlinedTextField dan digunakan untuk menangani perubahan isi input.

20. Kapan event - event tersebut dijalankan?

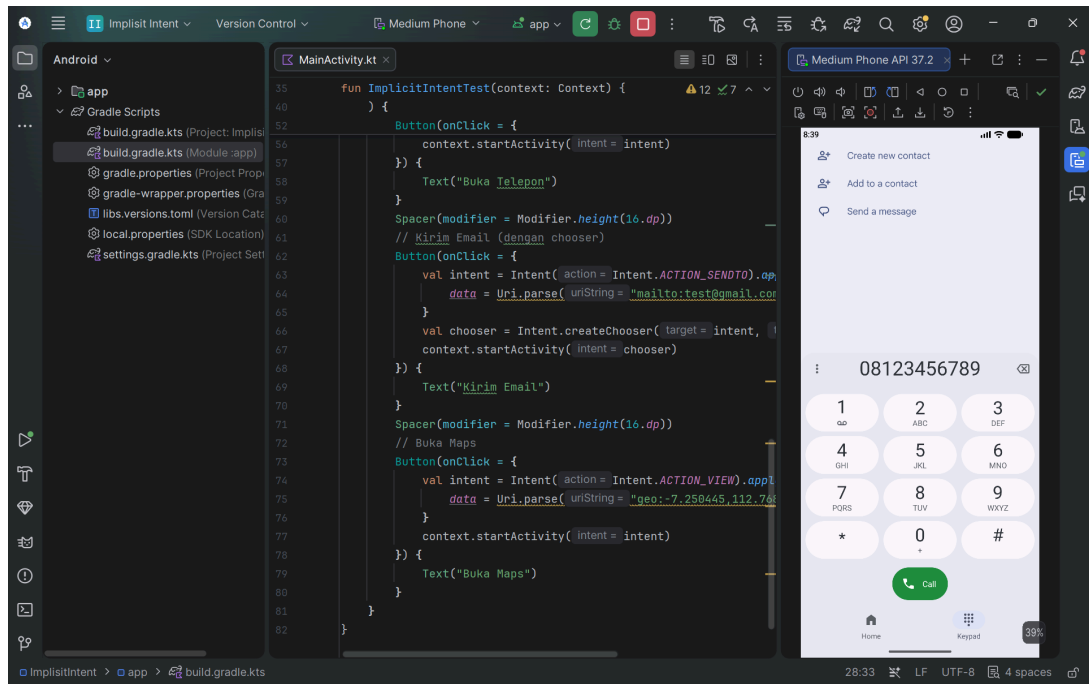
onClick dijalankan ketika pengguna menekan tombol Tambah. Event tersebut kemudian menambah nilai count. onChange dijalankan setiap kali nilai pada Text Field berubah, baik karena pengguna mengetik karakter baru, menghapus karakter, maupun mengubah teks yang sudah ada.

Implisit Intent

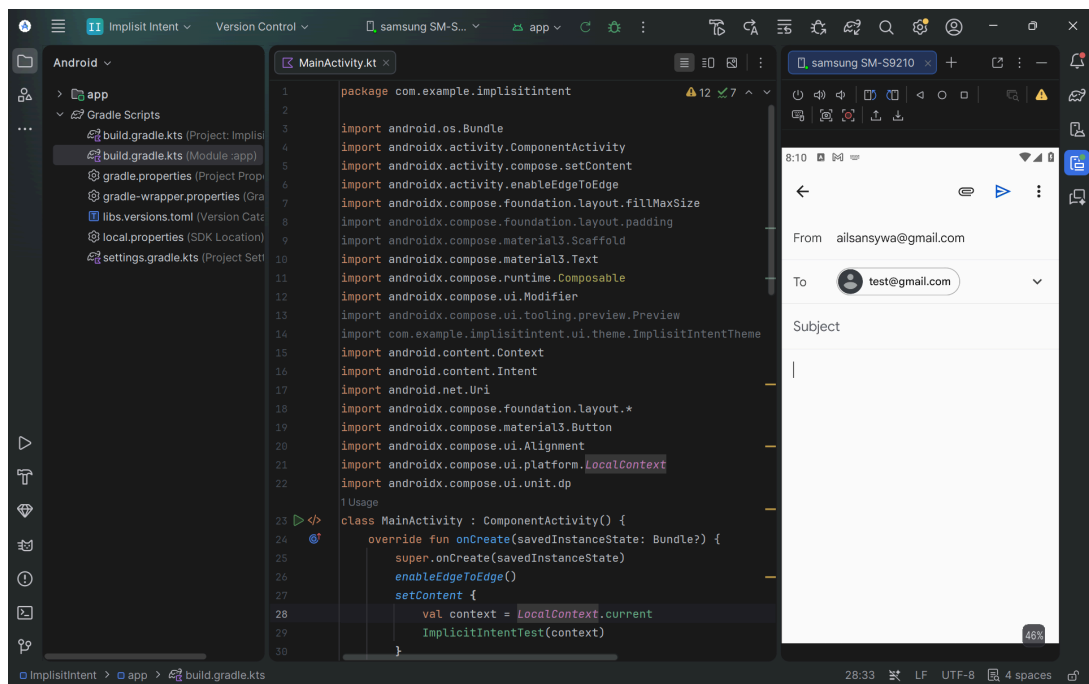
1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Lakukan pengujian pada setiap tombol:
2. Klik Buka Browser



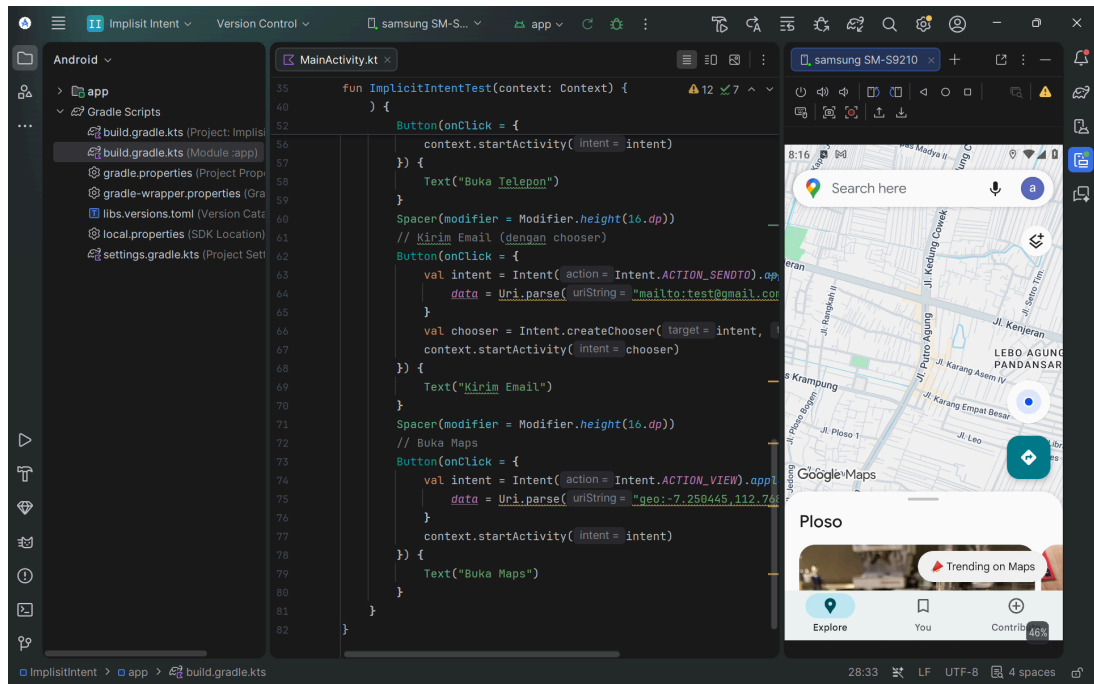
3. Klik Buka Telepon



4. Klik Kirim Email



5. Klik Buka Maps



6. Amati apa yang terjadi pada setiap aksi, kemudian ambil screenshot hasilnya

- 1) Buka Browser: sistem mencari aplikasi yang mampu menangani URL web dan kemudian membukanya.
- 2) Buka Telepon: sistem membuka aplikasi dialer dengan nomor “08123456789” sudah dimasukkan.
- 3) Kirim Email: sistem membuka aplikasi yang mampu menangani alamat email test@gmail.com. Pada kode juga digunakan chooser untuk memilih aplikasi yang sesuai.
- 4) Buka Maps: sistem membuka aplikasi pemetaan yang mampu menangani data lokasi geo:.

7. Pada kode yang digunakan, terdapat beberapa action seperti:

Intent.ACTION_VIEW

Intent.ACTION_DIAL

Intent.ACTION_SENDTO

Apa perbedaan fungsi dari masing-masing action intent tersebut?

- 1) Intent.ACTION_VIEW digunakan ketika aplikasi ingin meminta sistem membuka atau menampilkan suatu data. Dalam percobaan ini digunakan untuk membuka URL pada browser dan data lokasi pada aplikasi Maps.
- 2) Intent.ACTION_DIAL digunakan untuk membuka aplikasi telepon atau dialer dengan nomor tertentu. Action ini hanya mempersiapkan nomor pada dialer, bukan langsung melakukan panggilan.
- 3) Intent.ACTION_SENDTO digunakan ketika aplikasi ingin mengirim data kepada tujuan tertentu. Dalam percobaan ini action tersebut dipasangkan dengan URI mailto: sehingga sistem mencari aplikasi yang dapat menangani email.

12. Pada kode yang digunakan, terdapat beberapa data seperti:

data = Uri.parse("mailto:test@gmail.com")

```
data = Uri.parse("https://www.google.com")  
data = Uri.parse("geo:-7.250445,112.768845")
```

Apa fungsi data pada intent ?

“data” berfungsi memberikan informasi yang akan diproses oleh action. Informasi tersebut diberikan dalam bentuk URI. Contohnya:

- 1) Uri.parse("https://www.google.com") memberikan alamat website.
- 2) Uri.parse("mailto:test@gmail.com") memberikan alamat email tujuan.
- 3) Uri.parse("geo:-7.250445,112.768845") memberikan informasi berupa lokasi geografis.

Jadi, data membantu menjelaskan objek atau tujuan yang akan ditangani oleh Intent.

17. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android

Action menjelaskan tindakan yang ingin dilakukan, sedangkan Data menjelaskan informasi yang akan dikenai tindakan tersebut. Android kemudian melihat kombinasi keduanya untuk menentukan aplikasi yang sesuai. Contohnya:

- 1) ACTION_VIEW + https: dapat diarahkan ke browser.
- 2) ACTION_VIEW + geo: dapat diarahkan ke Maps.
- 3) ACTION_DIAL + tel: dapat diarahkan ke aplikasi telepon.
- 4) ACTION_SENDTO + mailto: dapat diarahkan ke aplikasi email.

Karena yang digunakan adalah implicit intent, program tidak menentukan nama aplikasi tujuan secara langsung. Sistem Android memilih aplikasi yang dapat menangani kombinasi action dan data tersebut.